

Fynd AI Engineering Intern Assessment - Final Report

Submitted by: Yogendra Wadkar | Date: December 7, 2025

Task 1: Rating Prediction via Prompting

1. Approach & Design Decisions

To address the challenge of categorizing Yelp reviews into ratings ranging from 1 to 5 stars, I created an experiment to evaluate three different prompting strategies. The objective was to identify which approach produces the greatest accuracy and structural dependability (valid JSON output).

Prompt Iterations :

- Method A: Zero-Shot (Baseline)
 - Design: I presented the review text and requested a JSON format output without providing any examples.
 - Hypothesis: This acts as a benchmark to evaluate how effectively the model comprehends sentiment without prior context.
- Method B: Few-Shot (Guided)
 - Design: I included two labelled instances (one positive and one negative) in the prompt to "educate" the model on the format and sentiment criteria.
 - Hypothesis: Offering examples would help align the model's rating system with the logic of the dataset.
- Method C: Chain-of-Thought (Reasoning)
 - Design: I directed the model to first identify positive and negative elements and then reason through them before assigning a final score.
 - Hypothesis: Encouraging the model to articulate an explanation prior to the rating would mitigate impulsive mistakes on complex reviews.

2. Evaluation Results:

The system was evaluated on a random sample of **60 reviews** from the Yelp dataset using the gemini-flash-latest model.

Prompting Strategy	Accuracy	JSON Validity	Key Observation
Chain-of-Thought	70.0%	100.0%	Best Performance. Reasoning through the review significantly improved precision.
Few-Shot	66.7%	100.0%	Strong baseline. Examples helped structure the output but occasionally missed nuance.
Zero-Shot	66.7%	100.0%	Surprisingly effective, but struggled to differentiate between 4-star and 5-star reviews.

3. System Behaviour & Challenges

Optimizing Sample Size (~200 vs. 60): The recommendation suggests using around 200 rows, but allows for sampling to enhance efficiency. I specifically opted to restrict the evaluation to 60 rows to preserve data integrity within the stringent Rate Limits of the LLM Free Tier. Using larger batches resulted in frequent API timeouts (HTTP 429), which threatened to taint the results with erroneous "0-star" failures. A sample size of 60 offered a statistically significant difference between the methods (Chain-of-Thought consistently outperformed the others) without destabilizing the evaluation process.

Managing API Rate Limits: To address the API limitations, I established a Robust Retry Mechanism that employs exponential backoff (with pauses of 20–50 seconds per error). This ensured that despite the optimization of the sample size, each row in the dataset was processed successfully without any loss of data.

JSON Validity: All three methods achieved complete validity for JSON. This was accomplished by using a stringent cleaning function (`clean_json`) that removed Markdown formatting (such as ````json` blocks) before parsing, guaranteeing that the downstream application would operate without crashing.

Final Thoughts: The Chain-of-Thought method is the preferred strategy. It provides roughly a 3.3% improvement in accuracy compared to other approaches by compelling the model to justify its choices, thus aligning more closely with human reasoning while remaining computationally efficient.

Task 2: AI-Powered Customer Feedback System

1. Methodology & Structure:

In response to the need for a dual-interface web application (User & Admin), I created a cohesive Single-Page Application (SPA) utilizing Streamlit. The platform is hosted on Hugging Face Spaces, providing worldwide accessibility and excellent uptime.

System Design:

- **Unified Architecture:** I adopted a "Monolith" design pattern, integrating both the Customer Feedback Form and the Admin Dashboard into one application (`app.py`). Access is managed through a Sidebar Navigation menu, which promotes efficient state management and removes the necessity of maintaining two distinct codebases.
- **AI Backend:** The platform utilizes Google Gemini 1.5 Flash for instantaneous text analysis. It manages two separate concurrent functions for each review received:
 1. **User Flow:** Produces a prompt, understanding reply to complete the customer feedback loop.
 2. **Admin Flow:** Assesses the sentiment to derive a concise 1-sentence summary and suggest a strategic course of action.
- **Data Persistence:** A simple CSV storage solution (`reviews_data.csv`) was created to save feedback between sessions, avoiding the complexity of an external SQL database and enhancing the prototype's performance and ease of use.

2. Deployment Challenges & Solutions

- **Environment Configuration:** My initial effort to containerize the application with raw Docker faced issues with port binding (Error: Connection Timed Out). I addressed this by transitioning to the Native Streamlit SDK on Hugging Face, which takes care of dependency installation (requirements.txt) and port exposure automatically.
- **State Management:** Streamlit executes the entire script with each interaction. To avoid unnecessary reloads of the Admin Dashboard, I implemented Streamlit's session state logic to maintain navigation stability while data is being written to the backend.

3. Final Deliverables (Live Links)

The application is live and publicly accessible. Both interfaces are hosted at the same URL for efficiency.

- **User Dashboard:** <https://huggingface.co/spaces/Naruto24Wadkar/Fynd-Internship-Task2>
- **Admin Dashboard:** <https://huggingface.co/spaces/Naruto24Wadkar/Fynd-Internship-Task2>
 - *(Note: The application is hosted on a free cloud tier. Please allow **5–10 seconds** for the initial cold boot. Once loaded, use the **Sidebar** on the left to toggle between User and Admin views.)*